

ZigBee CSS PHY Prerequisites

ZigBee CSS PHY Project — Prerequisites Guide

This document rebuilds, from zero, everything you need to understand the CSS PHY Transmitter reference document you uploaded. It assumes you remember almost nothing from comms/networks, and re-derives each concept in the order you'll actually meet it in the thesis text. At the end there's a concrete "where to start" roadmap for the physical (hardware) design.

1. Where does "PHY" sit in the big picture?

Any networked device is built as a stack of layers, each one hiding the complexity of the layer below it (this is the OSI model, but you only need the bottom two layers here):

Layer	Job	Example in this project
MAC (Medium Access Control)	Decides <i>when</i> to talk, handles addressing, retransmissions, acknowledgments	Not your focus — it just hands bytes to the PHY
PHY (Physical Layer)	Turns bytes into an actual radio waveform (and back)	This is your project

So: MAC says "send these 20 bytes to that device." PHY's job is to take those bytes and turn them into a sequence of voltage/frequency changes on an antenna — and, on the receive side, turn a noisy analog signal back into bytes.

IEEE 802.15.4 is the standard that defines several possible PHYs (different modulation schemes, frequency bands) for low-power, low-data-rate, short-range networks — this is the standard that ZigBee is built on top of. Your document is about one specific PHY option inside 802.15.4: **CSS (Chirp Spread Spectrum) at 2450 MHz**.

PAN (Personal Area Network) — just means a small local network (a room-scale mesh of sensors, for example). Not important for PHY design itself, just context for why the standard prioritizes low power over range/speed.

2. What a PHY layer actually does — the bit-to-waveform pipeline

Every digital radio transmitter follows the same generic recipe, and the whole rest of the document is just filling in the details of each step for CSS specifically:

bits → group bits into "symbols" → map each symbol to a signal shape → combine shapes over time → analog waveform

Two structural facts:

- **Symbol vs. bit:** A *symbol* is one “unit” transmitted at a time. It doesn’t have to be 1 bit — grouping multiple bits into one symbol lets you send more data per unit time (or, as in CSS, add redundancy for robustness). E.g., grouping 3 bits into 1 symbol and choosing from 8 possible waveforms is exactly what the “Symbol Mapper” block does for the 1 Mbps CSS rate.
 - **Packet structure:** A transmitted packet (PPDU) always has a **preamble** (a known pattern that lets the receiver lock onto timing/frequency — like a runway before landing), a **header** (says how long the payload is), and the **payload** (your actual data). This is universal to basically all digital comms, not just CSS.
-

3. Modulation — turning symbols into waveforms

This is the part most tied to your “digital communications” coursework (Gray coding, constellation diagrams) — here’s the refresher, tied directly to what’s used in CSS.

3.1 Constellations and I/Q

A **constellation diagram** is a 2D map of possible symbol values, plotted on two axes called **I** (in-phase) and **Q** (quadrature). Physically, I and Q are two sinusoidal carriers of the same frequency but 90° out of phase — because they’re orthogonal, you can pack two independent streams of information onto the same radio channel at the same time without them interfering. Any point on the I/Q plane corresponds to a specific amplitude+phase of the transmitted carrier.

QPSK (Quadrature Phase Shift Keying): 4 possible points on the constellation (see Fig 2-1 in your doc: (1,1), (-1,1), (-1,-1), (1,-1)), each representing 2 bits. This is exactly what the “QPSK Mapper” block does — I bits and Q bits get combined into one of these 4 complex symbols.

DQPSK (Differential QPSK): instead of encoding information as the *absolute* phase of the symbol, you encode it as the *change* in phase from the previous symbol. Why bother? Because a plain receiver doesn’t always know the absolute phase reference of the transmitter (there’s an ambiguity), but it *can* reliably measure the phase difference between consecutive received symbols. This is why equation (1) in your doc looks like $S_n = X_n * S_{n-4}$ — each new

symbol is the QPSK symbol multiplied by (not added to — multiplication rotates phase) the symbol 4 steps back. The “4” comes from the CSS-specific chirp structure explained in §5 below.

BPSK / ASK / GFSK / O-QPSK (mentioned in Table 1-2): other modulation options used by *other* 802.15.4 PHYs (not CSS) — you don’t need their details, just recognize CSS is one option among many, chosen for range/robustness over raw speed.

3.2 Why “block coding” / the Symbol Mapper exists

Before modulation, CSS passes bits through a **Symbol Mapper**: e.g. for the 1 Mbps rate, every 3 bits get mapped to a 4-bit codeword chosen from a fixed table (Walsh-Hadamard codes). This is **error-control coding**: by adding redundancy (3 bits → 4 bits) using codewords that are maximally “different” from each other (Walsh-Hadamard codes are mutually orthogonal), the receiver can tolerate more noise before misreading a symbol — a bit flip is less likely to turn one valid codeword into another. This trades bandwidth/rate for robustness, which is the whole philosophy of CSS (see §1.2.1 “Advantages” in your doc).

3.3 Interleaving

The **Interleaver** just re-orders bits/groups before transmission (see the G0→G13 permutation table). Purpose: if a burst of interference corrupts several *consecutive* transmitted bits, interleaving spreads those originally-adjacent bits apart in time, so after de-interleaving at the receiver, the errors are scattered thinly instead of clustered — much easier for the error-correcting code to fix scattered single errors than one dense burst.

4. Spread Spectrum — the “SS” in CSS

Core idea: instead of squeezing your signal into the minimum bandwidth needed, deliberately spread it across a much wider bandwidth than the data rate requires.

Why this helps: - **Interference immunity**: narrowband noise/jammers only wipe out a small slice of your wide signal; most of your signal energy survives. - **Coexistence**: multiple systems can share the same frequency band without coordinating, because each spread signal looks like low-level noise to the others. - **Multipath robustness**: in an indoor/urban environment, your radio wave bounces off walls and arrives at the receiver as several delayed copies of itself. A wideband, chirp-like signal is much easier to time-align and combine correctly than a narrowband one — this is the actual technical reason CSS is chosen for “long-range or mobile” links per §1.2.1.

DSSS (Direct Sequence Spread Spectrum) — the more common cousin (Wi-Fi’s original PHY, GPS): you XOR your data with a fast pseudorandom code to widen the spectrum. CSS uses a *different* spreading mechanism — chirps instead of a PN code — but the underlying goal (spread the energy over a wide bandwidth on purpose) is identical.

Processing/spreading gain: how much wider you make the signal vs. its native bandwidth. Your document calls this the “spread factor $\nu = B/R$ ” (bandwidth divided by data rate) — the bigger this ratio, the more robust but the more radio spectrum you consume for the same data rate.

5. Chirps — the actual CSS mechanism

A **chirp** is a signal whose frequency sweeps linearly up or down over time (think of the sound of a bird chirp, or radar “pings”).

Mathematically, instead of a fixed-frequency sinusoid $\cos(w_0 * t)$, the phase argument includes a t^2 term (see Table 1-7’s chirp expression: $w_0 * t + u * t^2 / 2$) — differentiating phase with respect to time gives you instantaneous frequency, and a t^2 phase term means frequency itself is *ramping linearly with time*. That’s what “chirp” mathematically means.

Why chirps specifically? - A chirp inherently occupies the *entire* available bandwidth over its duration (it sweeps through every frequency in the band), which is exactly the “spread the signal” goal from §4 — for free, with a simple, well-understood waveform. - Chirps are easy to generate and, importantly, easy to *detect* even with a lot of noise, using **matched filtering** (correlating the received signal against a copy of the expected chirp) — this is the same principle radar uses. - Time-Bandwidth Product (BT): a single number describing how “spread” a chirp is (duration $\times$ bandwidth). Bigger BT = more resistant to noise, at the cost of taking longer to transmit each symbol.

CSK (Chirp Shift Keying): this is the actual modulation trick in §2.1.3 step 8. Instead of one fixed chirp shape, the transmitter picks **which one of 4 possible chirp sequences** (indexed by $m = 1, 2, 3, 4$) to send, and *that choice itself* carries information (via the DQPSK symbol driving it) — analogous to how FSK picks one of several frequencies, except here it’s “pick one of several chirp patterns.”

Subchirps: each transmitted chirp sequence isn’t one continuous sweep — it’s built from 4 shorter “subchirp” segments ($k=0..3$, Table 2-3/2-4), each with its own direction (increasing or decreasing frequency) and centre frequency, and each individually modulated by one DQPSK symbol. This is why 4 consecutive DQPSK symbols map onto 1 chirp sequence — and this is *exactly* the origin of the “delay by 4” in the DQPSK equation you saw in §3.1: the differential coding scheme is deliberately matched to the fact that a full chirp/DQPSK group spans 4 symbols.

Raised cosine windowing (equation 3, Fig 2-3): a smooth tapering function applied to the edges of each subchirp pulse. Purpose: an abruptly-starting/stopping pulse has a very wide, “splattered” frequency spectrum (a sharp edge in time = infinite frequency content, from Fourier theory). Tapering the edges smoothly keeps the transmitted energy contained within the intended channel bandwidth instead of leaking into neighboring channels.

6. From analog theory to hardware — the concepts bridging Ch.2 and Ch.3

Your document deliberately builds a **floating-point (MATLAB) model first**, then converts it to **fixed-point**, then to **HDL/FPGA**. Here's why each concept matters, since you already know HDL/FPGA flow well from your other projects — this is really just DSP-specific new vocabulary:

- **Floating point vs. fixed point:** floating point (what MATLAB uses by default) represents huge dynamic range with full precision but is expensive in hardware (needs complex ALUs). Fixed point uses a fixed number of bits with an implied decimal point (e.g., “8 bits, resolution R”) — cheap in FPGA logic, but you need to verify you haven't thrown away too much precision.
- **MSE (Mean Square Error):** the metric your doc uses (Fig 3-2) to decide *how many bits* (R) are enough for the fixed-point chirp samples — comparing the fixed-point rounded output against the “exact” floating-point reference, per equation in §3.1. This is the standard way to pick word-widths in any DSP hardware design: sweep bit-width, plot error, stop increasing bits once error is acceptably below the spec (0.005 here, giving R=5 as chosen).
- **I/Q as two real-valued signals:** hardware can't directly manipulate a complex number — every “complex” CSS symbol from §3 above is really carried on **two parallel real datapaths** (Tx_real / Tx_imag in Table 3-1). Any complex multiply (like the DQPSK multiply, or CSK modulation) becomes 4 real multiplies + 2 adds in hardware — worth knowing before you start writing the multiplier block.
- **ROM-based lookup tables:** notice the architecture (Fig 3-3) implements the Symbol Mapper and the CSS chirp sequences as ROMs (ROM = pre-computed constants), not as real-time trig calculations — this is a standard DSP-hardware trick: since the chirp waveforms are fixed and known in advance, it's cheaper to precompute all 152 samples × 4 chirp indices once (in MATLAB) and store them as constants than to calculate sin/cos on the fly. (If you ever *do* need to generate sinusoids on the fly in hardware, that's what a **CORDIC** algorithm — referenced [16] in your doc — is for; worth a quick read if the project later needs a receiver with real-time trig, but not needed for this transmitter since it's ROM-based.)

7. Quick-reference glossary

Term	One-line meaning
PPDU	The full transmitted packet at the physical layer (preamble + header + payload)
PSDU	Just the payload bytes, before PHY wraps them into a PPDU
SHR / SFD	Sync Header / Start-of-Frame Delimiter — lets receiver find the start of real data
PHR	PHY Header — carries the payload length
Symbol	One “unit” of transmission, can carry multiple bits The two orthogonal carrier components used to represent

I/Q	any modulated signal as a complex number
QPSK / DQPSK	4-point constellation modulation; “D” = encode phase <i>change</i> , not absolute phase
Chirp / LFM	A pulse whose frequency sweeps linearly over time
CSK	Choosing which chirp pattern to send = the actual information-carrying step
Spreading factor (ν)	Bandwidth $\div$ data rate — how “spread out” the signal is
BT product	Duration $\times$ bandwidth of a chirp — bigger = more noise-resistant
MSE	Metric used to size fixed-point word-widths against a floating-point reference

8. Where to start on the physical (hardware) design

Given you already have strong HDL/FPGA/testbench experience (FSMs, FIFOs, AXI, synthesis flows), the *new* material here is DSP-specific, not HDL-specific. Recommended order, mirroring the document’s own structure:

1. **Re-derive/verify the floating-point model in MATLAB first**, even though it’s already described — build each block (§2.1.3, steps 1-8) as a separate MATLAB function and check outputs stage-by-stage against the equations. Don’t skip this: it becomes your “golden reference” for every later HDL testbench, exactly the workflow described in §3.1.2 (MATLAB TestBench / EDA Simulator Link).
2. **Start hardware design with the simplest, most familiar blocks** — these use skills you already have:
 - Zero Padding, Demux, Form PPDU — pure control/muxing logic, essentially FSM + counter work like your bank queue monitor project.
 - Symbol Mapper, Interleaver — ROM lookup + fixed permutation wiring, similar in spirit to your ROM-based work.
3. **Then QPSK Mapper and DQPSK Coding** — this is where you apply the complex-multiply-as-4-real-multiplies concept from §6, plus the 4-deep delay line. Good place to also practice the fixed-point MSE-sizing exercise (§3.1 of the doc) on a small scale before tackling the chirp block.
4. **Save CSK Generation / chirp sequence ROM for last** — it’s explicitly called out in the doc as “the key process” and “the source of the floating point,” i.e., the hardest/most novel block. By the time you reach it you’ll already have your fixed-point verification methodology proven on the earlier, simpler blocks, and you’ll just be applying it to a bigger ROM + multiplier chain.
5. **Testbench strategy**: reuse the MATLAB-as-testbench flow (§3.1.2 of the doc) per block, not just at the end — this is the same instinct you already showed on your FIFO/AXI projects (unit-test each module, then integrate), just adapted to compare against a

MATLAB floating-point reference instead of a hand-written expected-value table.

This lets you build momentum on blocks that map directly onto skills you already have, and defer the genuinely new DSP concepts (chirps, spreading, fixed-point sizing) to later stages — by which point the earlier blocks will have given you a working pipeline to plug them into.